

Service in K8s (Load Balancer, Service Discovery):

In Docker Compose Service is to create multiple containers from a single image.

Why Service?

Helps to access application running in a single/group of pods.

What is a Service and when we need it? 🤔

- ▶ Each Pod has its own IP address

❌ Pods are ephemeral - are destroyed frequently!



ip address

new IP



ip address

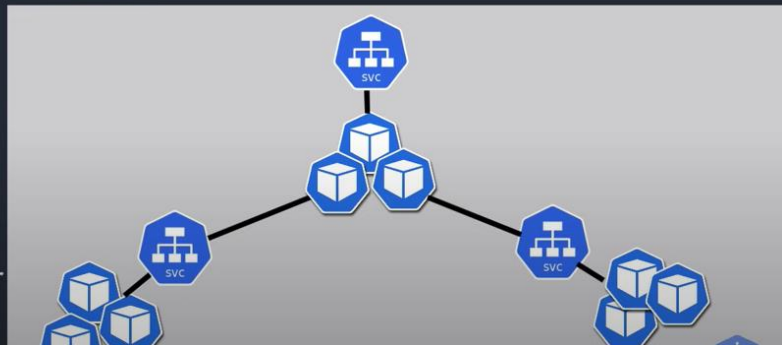
What is a Service and when we need it? 🤔

- ▶ Each Pod has its own IP address

❌ Pods are ephemeral - are destroyed frequently!

- ▶ Service:

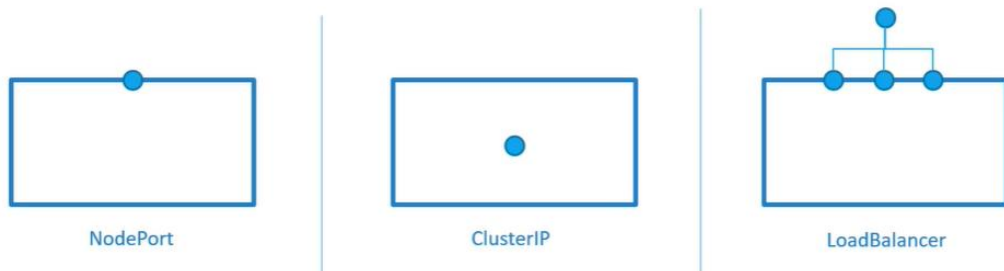
- ✓ stable IP address
- ✓ loadbalancing
- ✓ loose coupling
- ✓ within & outside cluster



- In K8s service enable the communication between the Pods and also going to acts as a Load Balancer.
- Service is responsible for routing the request behind the containers.
- Services acts as an Internal LoadBalancer as well as forwarding the request into all backend pods.

Services: (Virtual entity not a physical SVC IP is virtual IP, API server assigns service a Virtual IP).

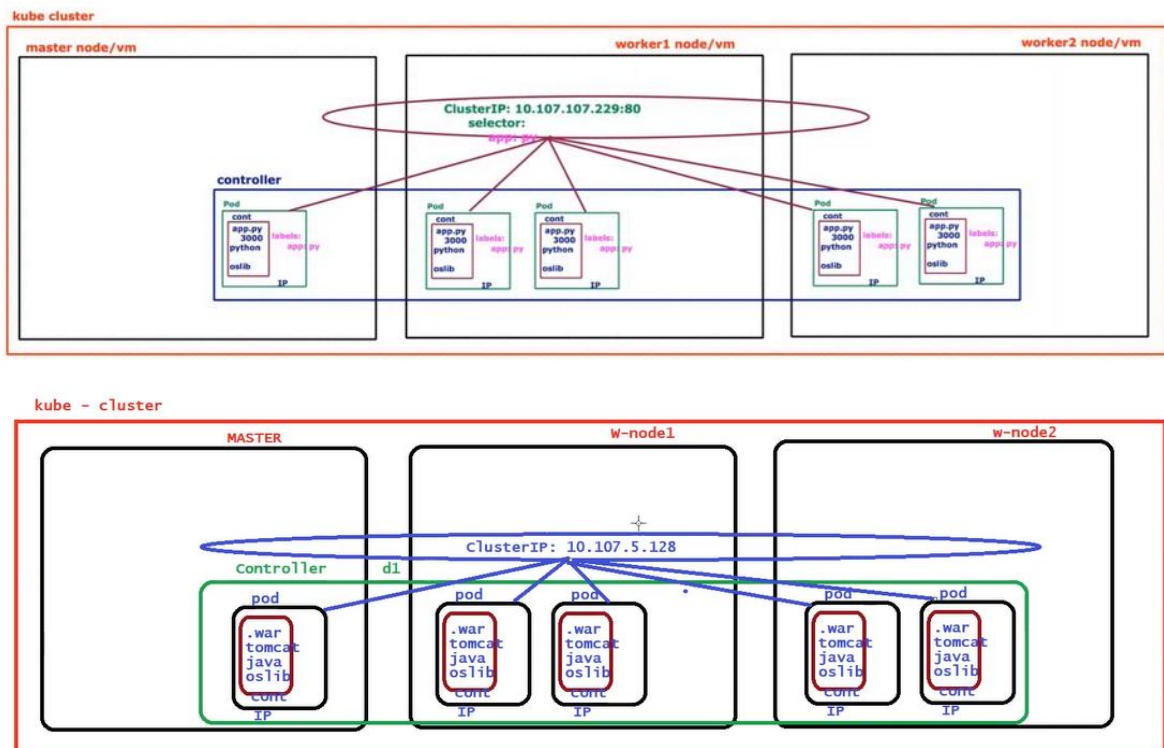
Types of Services:



ClusterIP: (Internal Service)

NodePort: Access from outside the cluster

LoadBalancer: Cloud Vendor Specific



How to access Pod from outside the cluster (browser)?

When we create a Pod, K8s assigns IP address to the Pod, Pod IP address not accessible from outside the world, because this IP address from the weave network, weave network we installed within the K8s that is only for K8s communication, internal communication of the K8s. We can't use it from outside the world.

Within Docker we use `-p` or `-P` to publish the port number, `nodeport(serviceport)` is always created in case of docker swarm.

We need to create a service, service is like a proxy to your pod, sender of request to your pod. Service will have its own IP address, then by using IP of the service can access the Pod.

What is the backend of this service?

In k8s when we create a Service Object, internally it also creates endpoint object

Service

```
apiVersion: v1
kind: Service
metadata:
  name: node-app-service
spec:
  type: NodePort
  selector:
    app: node-app
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30464
```

Endpoints

```
apiVersion: v1
kind: Endpoints
metadata:
  name: node-app-service
subsets:
- ports:
  - port: 3000
- addresses:
  - ip: 10.0.0.1
  - ip: 10.0.0.2
  - ip: 10.0.0.3
```

```
[k8s-services]$ kubectl get endpoints
NAME                ENDPOINTS                                AGE
kubernetes           172.104.231.137:6443                    15m
mongodb-service     10.2.1.4:27017,10.2.1.5:27017          5m27s
```

K8s creates Endpoint object

- same name as Service

- keeps track of, which Pods
are the members/endpoints of the Service

Kubectl describe service <name of service>

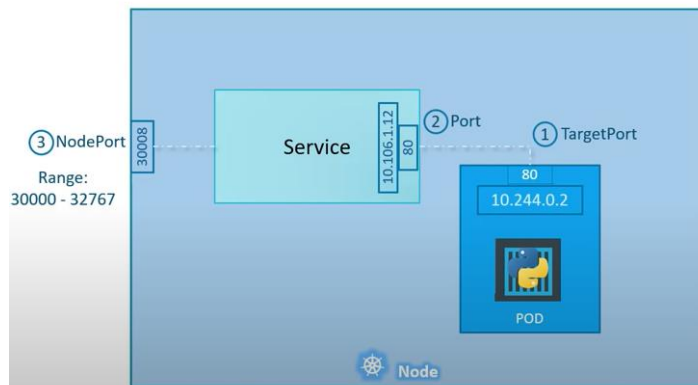
kubectl get ep <name of service>

// endpoints

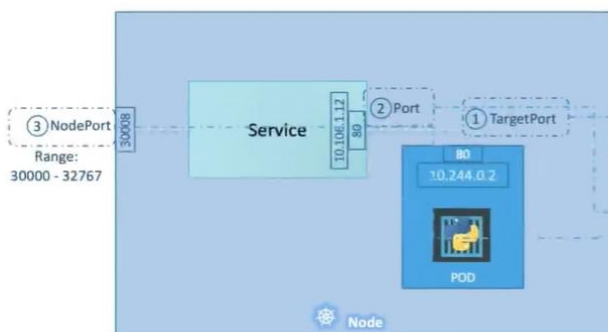
kubectl get pods -o wide

NodePort

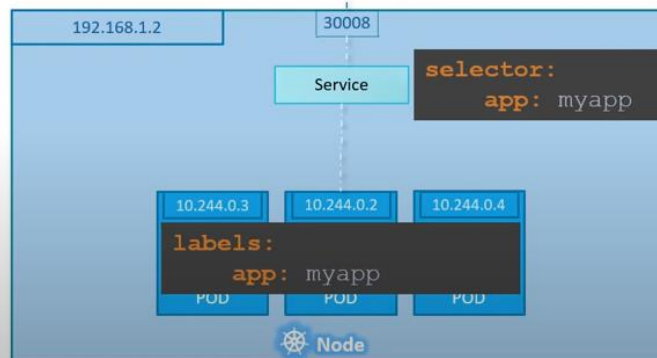
Use Case: You have deployed your web app on to Kubernetes cluster, now you need to expose it outside world on the internet.



```
service-definition.yml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      *port: 80
      nodePort: 30008
```

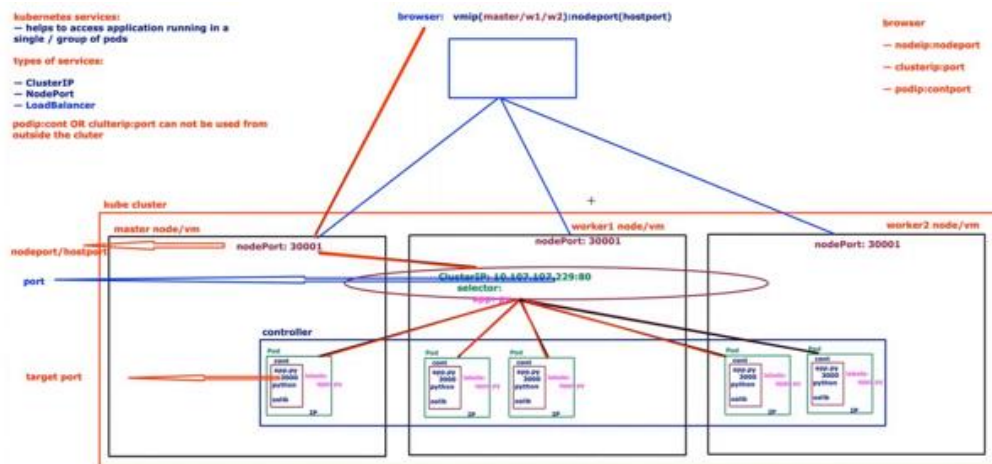
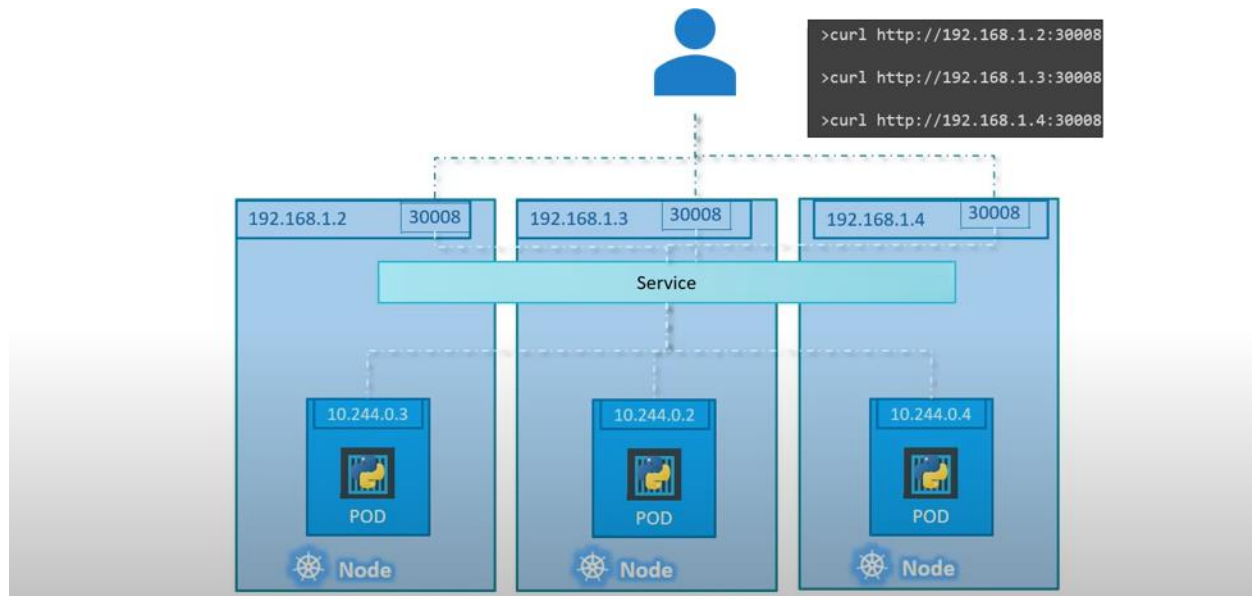


```
service-definition.yml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      *port: 80
      nodePort: 30008
```



```
selector:
  app: myapp
```

```
labels:
  app: myapp
```



k expose deploy <deployment name> --name=<service name> --port=<service port> --target-port=<container port> --type=NodePort

k expose deploy foo --name=foo --port=6262 --target-port=8080

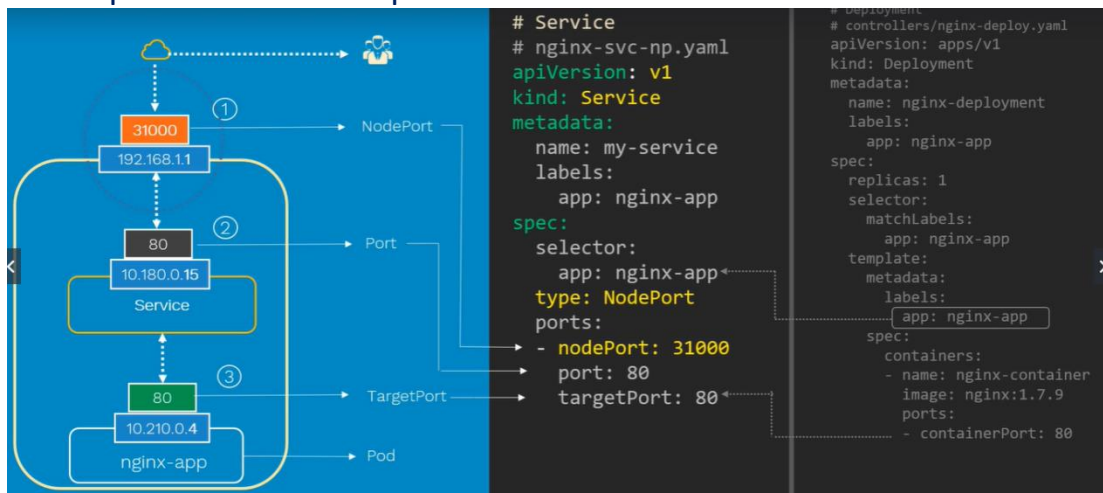
```

root@ip-172-31-23-45:~# kubectl get svc --show-labels
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE    LABELS
kubernetes           ClusterIP   10.96.0.1      <none>         443/TCP          74m    component=apiserver,pro
der=kubernetes
mysvc                NodePort    10.97.63.214   <none>         8080:31773/TCP   14m    <none>
mysvc1               NodePort    10.102.239.65  <none>         3000:30168/TCP   11m    <none>
mysvc2               NodePort    10.100.37.49   <none>         8080:32082/TCP   46s    <none>
root@ip-172-31-23-45:~# kubectl describe svc mysvc2
Name:
Namespace:
Labels:
Annotations:
Selector:
Type:
IP:
Port:
TargetPort:
NodePort:
Endpoints:
Session Affinity:
External Traffic Policy:
Events:
mysvc2
default
<none>
<none>
NodePort
10.100.37.49
<unset> 8080/TCP
8080/TCP
<unset> 32082/TCP
10.32.0.3:8080,10.32.0.4:8080,10.40.0.3:8080
None
Cluster
<none>

```

Limitation:

In real scenario we will never share our node ip to end user, is good for development but not for production.



Port Types:

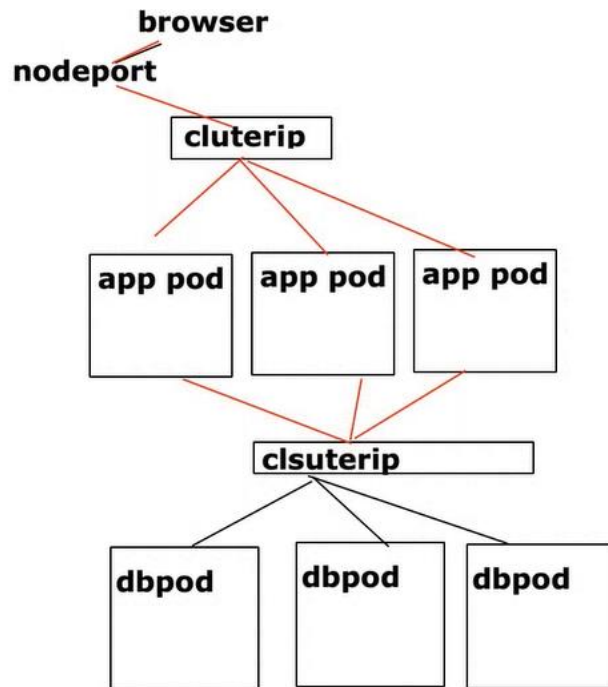
nodePort: port on the node, where pod is running (30000-32767)

port: on Service: This is the port on the service itself.

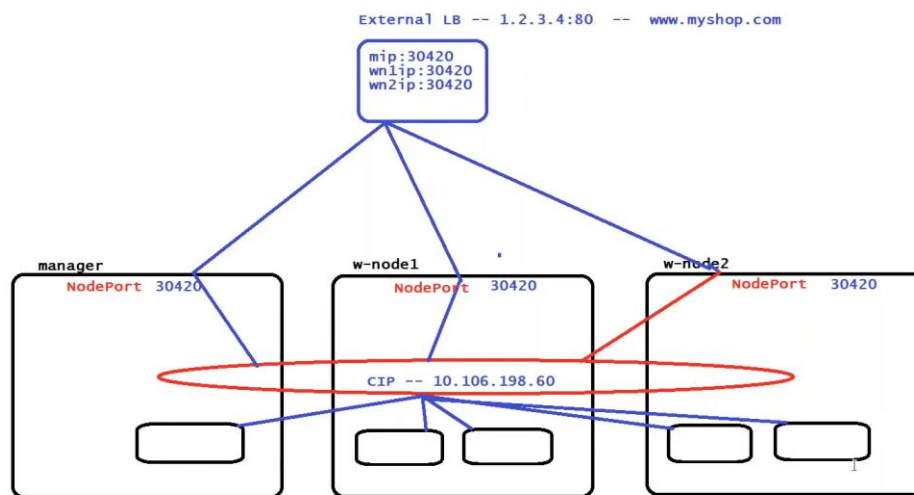
targetPort: This is the port on the actual pod where web application is actually running.

To access from browser first we need to first access node then from node to access cluster and then from cluster to Pod.

Service publish the nodePort, using the nodePort can access Pod from outside the world. Service forwards the request to the Pod.



LoadBalancer:



Kubernetes does not provide external LoadBalancer, **type:** LoadBalancer will work only in case of External LoadBalancer (If we create a LB service in our local environment without configuring external LB, service will not work), we can take advantage of cloud providers (GCP, AWS) e.g.

GCP->Kubernetes Engine->Clusters-> create cluster

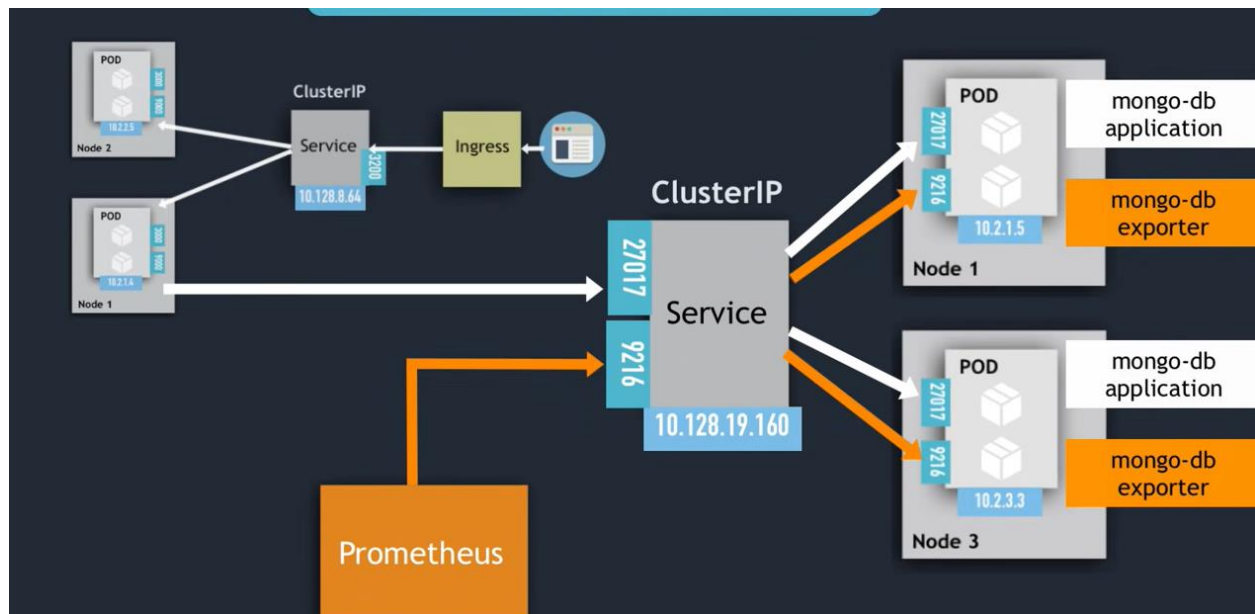
External LoadBalancer: By default, provided by all cloud provider at time of cluster creation e.g., GKS, EKS, PKS (Cluster provided by vmWare) at time of Object creation. LoadBalancer registered with available nodes in the cluster. This LB have single IP or DNS name.

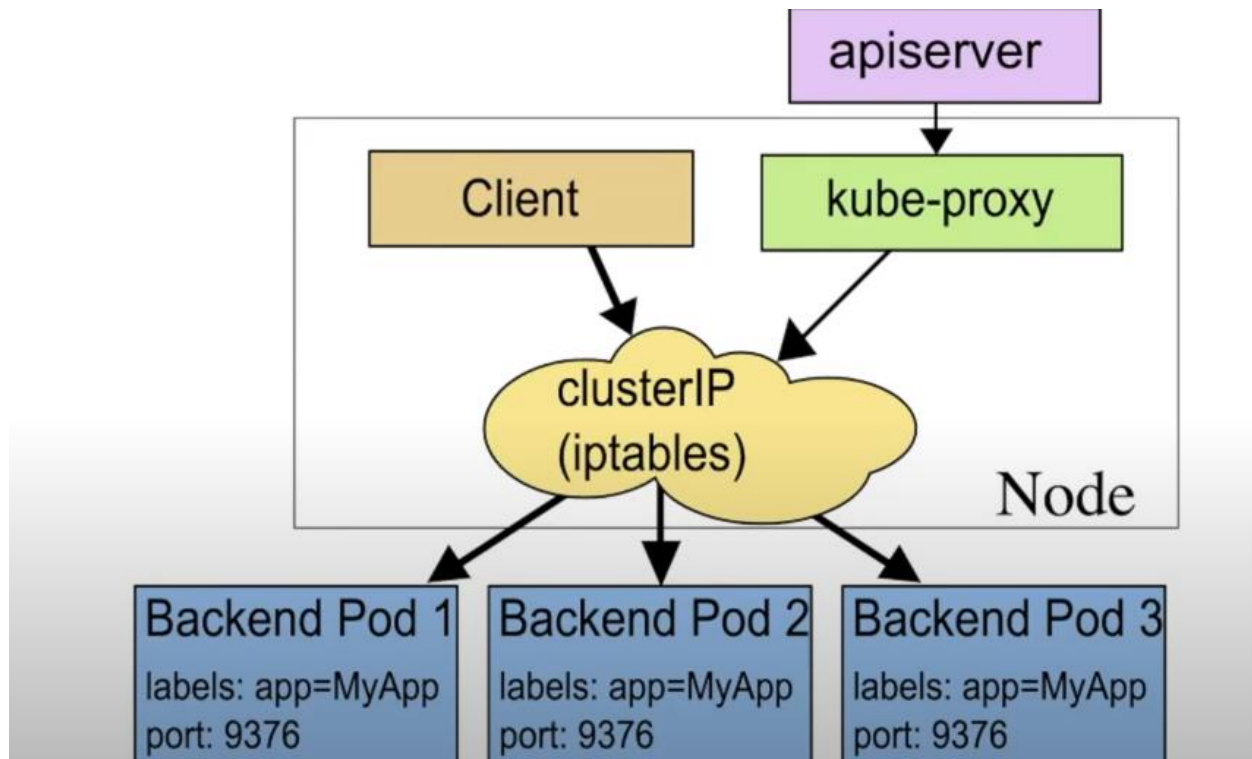
Cloud Vendor automatically creates LoadBalancer on the backend when we create a service of type LoadBalancer

LoadBalancer is not a cheap it's very costly, think before creating LoadBalancer.

When we have a cluster in cloud which provides default external LoadBalancer, and when we create an object of type LoadBalancer, cloud provider automatically will associate external load balancer with all the nodes of the cluster.

UC:





How to create GKS cluster?

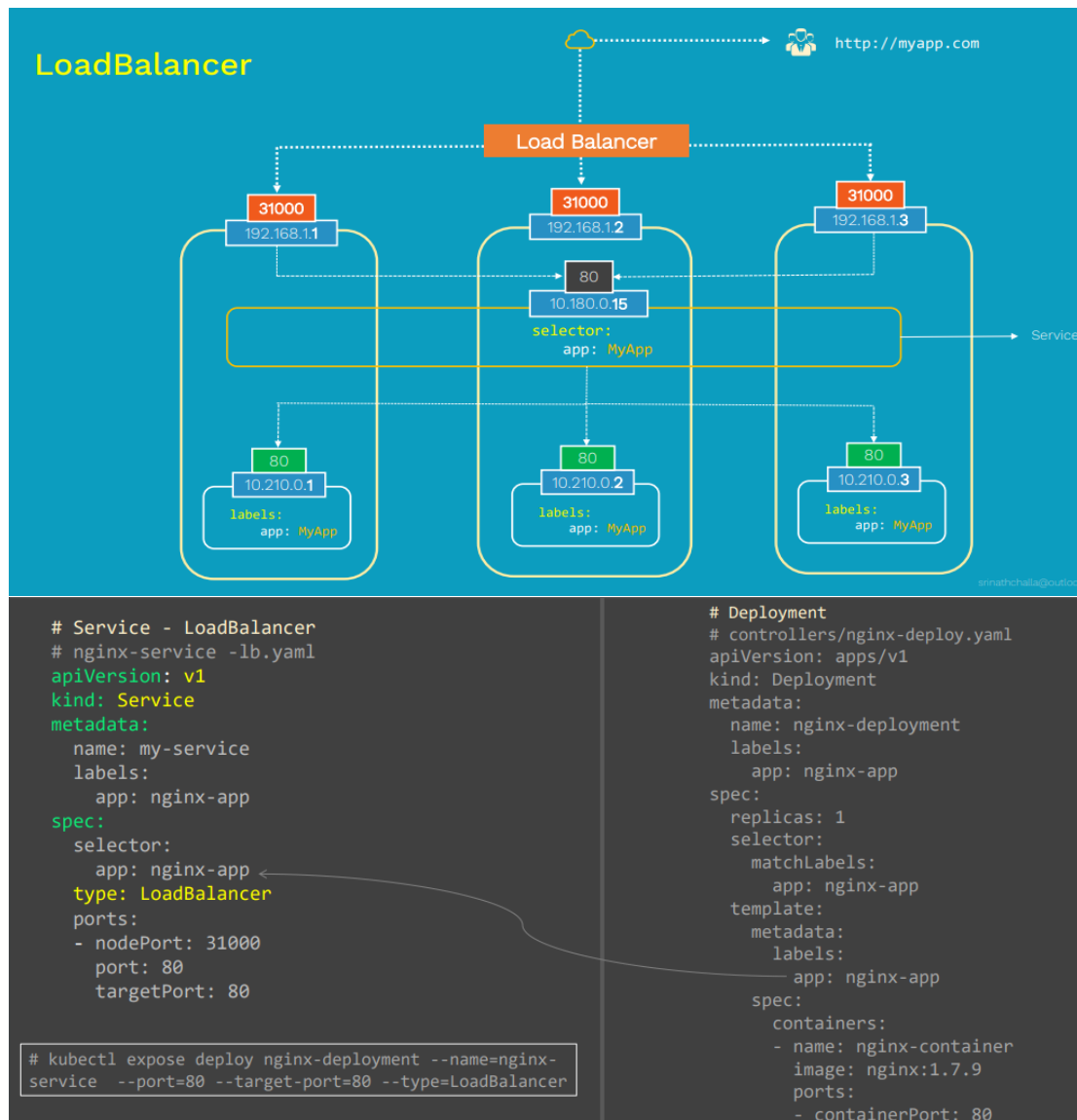
Home->Kubernetes Engine->Clusters-> create cluster + add Nodes

Home->Network Services-> Load Balancing // No LoadBalancer exist

`kubectl apply -f <loadbalancer.yaml>` // Internally creates ClusterIP, NodePort and External IP and LB

`kubectl get svc`

External LoadBalancer automatically created and map with the NodePort.



Step1: When you access the application from external world request access through any **node** of the cluster.

Kube-proxy in iptables proxy mode

- kube-proxy watches the Kubernetes control plane for Services and Endpoints
- For each Service, it installs iptables rules
- iptable rules capture Service's clusterIP and port, and redirect that traffic to one of the Service's backend sets
- For each Endpoint object, it installs iptables rules which select a backend Pod
- Kube-proxy relies on readiness probe to determine if a backing pod is healthy

while true; do curl <ip> <port>; sleep 2; echo "">; done

In k8s the only resource, which physical run on a virtual machine is are Pods, apart from that any other resource in k8s are just configuration they are not running executable.

Service is a cluster level resource, it's just a configuration, and it's always point to the pods on the top of which service is created. Service does the following two things when it's created. It registers themselves inside CoreDNS Pod. (Virtual IP assigned to service), Service is just only the configurations it can never be crashed, it is just a combination of firewall rules and the IP tables.

Service is a configuration in IP tables.

A service is a grouping of pods that are running on the cluster.

```
kind: Service    # k api-resources
apiVersion: v1   # k explain < resource>
metadata:
spec:
  type:
  ports:
    - port:      # service port
      targetPort: # container port
      nodePort:   # nodePort
  selector:
    app: front-end
```

UC1:

- You want to have an external database cluster in production, but in your test environment you use your own databases.
- You want to point your Service to a Service in a different Namespace or on another cluster
- You are migrating a workload to Kubernetes. While evaluating the approach, you run only a portion of your backends in Kubernetes.

In any of these scenarios you can define a Service **without a Pod selector**. For example:

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: my-service
spec:
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Because this Service has no selector, the corresponding Endpoints object is not created automatically. You can manually map the Service to the network address and port where it's running, by adding an Endpoints object manually:

```
apiVersion: v1
kind: Endpoints
metadata:
  name: my-service
subsets:
  - addresses:
    - ip: 192.0.2.42
    ports:
    - port: 9376
```

Accessing a Service without a selector works the same as if it had a selector. In the example above, traffic is routed to the single endpoint defined in the YAML: 192.0.2.42:9376 (TCP).

UC1.1: External Service (DB outside the K8s Cluster)

We may have different Instances for different environments



10.240.0.4:5000



localhost:5000

External Services with IP addresses

Internal IP address

☐	☒	mongodb-1-server-1	us-west1-b	10.240.0.4	35.185.--
--------------------------	-------------------------------------	--------------------	------------	------------	-----------

Create Kubernetes

```
kind: Service
apiVersion: v1
metadata:
  name: mongo
Spec:
  type: ClusterIP
  ports:
    - port: 27017
      targetPort: 27017
```

Create Endpoints

```
kind: Endpoints
apiVersion: v1
metadata:
  name: mongo
subsets:
  - addresses:
      - ip: 10.240.0.4
    ports:
      - port: 27017
```


UC3:

An **ExternalName** Service is a special case of Service that does not have selectors and uses DNS names instead. Services of type **ExternalName** map a Service to a **DNS** name, not to a typical selector such as my-service or cassandra. You specify these Services

with the spec.externalName parameter. This Service definition, for example, maps the my-service Service in the prod namespace to my.database.example.com:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

UC4:

If there are external IPs that route to one or more cluster nodes, Kubernetes Services can be exposed on those external IPs. Traffic that ingresses into the cluster with the external IP (as destination IP), on the Service port, will be routed to one of the Service endpoints.

externalIPs are not managed by Kubernetes and are the responsibility of the cluster administrator.

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
  externalIPs:
    - 80.11.12.10
```